

Oefeningen API

Refactoren bestaande code

JokeExample.vue

```
vue

<script setup>
import { ref } from "vue";

const joke = ref(null);
const loading = ref(false);

async function getJoke() {
  loading.value = true;
  joke.value = null;

  const res = await
fetch("https://api.chucknorris.io/jokes/random");
  const data = await res.json();

  loading.value = false;
  joke.value = data.value;
}
</script>

<template>
  <button @click="getJoke">Fetch Chuck Norris
Joke</button>
```

```

<p v-if="loading">Loading joke...</p>
<p v-else-if="joke">{{ joke }}</p>
</template>

```

Neem de code over en refactor zodat:

- Er een `error` -state wordt bijgehouden in een `ref` -variabele.
- Er een `try...catch...finally` -blok wordt gebruikt bij het uitvoeren van de `fetch`.
- In het `catch` -blok wordt de `error` -state bijgewerkt met een foutmelding.
- In het `finally` -blok wordt de `loading` -state bijgewerkt zodat deze altijd correct wordt ingesteld, ongeacht of de fetch is geslaagd of mislukt.
- Er een `disabled` -attribuut dynamisch wordt gekoppeld aan de knop, zodat de knop niet kan worden aangeklikt terwijl de `fetch` bezig is.

Throttling - netwerk trager maken

In de netwerktab van de `developer tools` in de webbrowser kan de netwerksnelheid vertraagd worden (bijvoorbeeld `3G`) om het laden van de grap trager te maken. Zo kan er beter getest worden hoe de `loading` -state wordt weergegeven. Op deze manier kan er ook getest worden hoe gebruikers op een langzamere verbinding de applicatie ervaren.

▼ Oplossing

```

<script setup>
import { ref } from "vue";
const joke = ref(null);
const loading = ref(false);
const error = ref(null);

async function getJoke() {
  loading.value = true;
  joke.value = null;

```

vue

```

error.value = null;

try {
  const res = await
fetch("https://api.chucknorris.io/jokes/random");
  const data = await res.json();
  joke.value = data.value;
} catch (err) {
  error.value = "Fout bij het ophalen van de grap: " +
err.message;
} finally {
  loading.value = false;
}
}
</script>

<template>
  <button @click="getJoke" :disabled="loading">Fetch Chuck
Norris Joke</button>

  <p v-if="loading">Loading joke...</p>
  <p v-else-if="error">{{ error }}</p>
  <p v-else-if="joke">{{ joke }}</p>
</template>

```

Dit werkt, maar de `fetch` call - de `joke` -data - wordt nu beheerd in de component zelf. Stel dat dezelfde `joke` -data ook op een andere pagina opgehaald moet worden, dan zou de `fetch` -logica in meerdere componenten gekopieerd moeten worden. Dit is niet optimaal.

Refactor de code verder zodat:

- De `fetch` -logica en `joke` -data worden beheerd in een Pinia store in `stores/joke.js`.
- De component de `joke` -data, `loading` -state, `error` -state en `getJoke` -functie uit de Pinia store haalt en gebruikt.

▼ Oplossing

`stores/joke.js`

js

```
import { defineStore } from "pinia";
import { ref } from "vue";

export const useJokeStore = defineStore("joke", () => {
  const joke = ref(null);
  const loading = ref(false);
  const error = ref(null);

  async function getJoke() {
    loading.value = true;
    joke.value = null;
    error.value = null;

    try {
      const res = await
fetch("https://api.chucknorris.io/jokes/random");
      const data = await res.json();
      joke.value = data.value;
    } catch (err) {
      error.value = "Fout bij het ophalen van de grap: " +
err.message;
    } finally {
      loading.value = false;
    }
  }

  return { joke, loading, error, getJoke };
});
```

JokeExample.vue

vue

```
<script setup>
import { useJokeStore } from "@stores/joke";
const jokeStore = useJokeStore();
</script>

<template>
  <button @click="jokeStore.getJoke"
:disabled="jokeStore.loading">
    Fetch Chuck Norris Joke
  </button>
</template>
```

```
<p v-if="jokeStore.loading">Loading joke...</p>
<p v-else-if="jokeStore.error">{{ jokeStore.error }}</p>
<p v-else-if="jokeStore.joke">{{ jokeStore.joke }}</p>
</template>
```

Conventie

Gebruik vanaf nu altijd een Pinia store om API-calls en data te beheren in plaats van dit rechtstreeks in de component te doen.

CRUD oefening

CRUD staat voor **C**reate, **R**ead, **U**ppdate en **D**eleete. Het zijn de vier basisbewerkingen die op data in een applicatie kunnen worden uitgevoerd.

Tot nu toe is enkel data opgehaald (Read) via een API met de `fetch` -methode. Een `fetch` request wordt standaard als een `GET` request uitgevoerd. De [documentatie van jsonplaceholder](#) kan worden geraadpleegd om te zien hoe ook `POST`, `PUT` en `DELETE` requests kunnen worden uitgevoerd.

Fake API

`jsonplaceholder` is een fake online REST API die wordt gebruikt voor test- en prototypingdoeleinden. Dit betekent dat hoewel `POST`, `PUT` en `DELETE` requests kunnen worden verstuurd, de data op de server niet echt wordt aangepast. Dezelfde `responses` zoals beschreven in de documentatie zullen altijd worden teruggegeven door de API.

In een echte applicatie zal de backend server de data wel effectief wijzigen in de database.

Store

Maak een Pinia store `stores/todo.js` die de volgende functionaliteiten bevat:

Let op

De `todos` -state zal na het uitsturen van een `POST`, `PUT` of `DELETE` request zelf moeten worden bijgewerkt, omdat de data in de backend door `jsonplaceholder` niet echt wordt aangepast. Dit kan worden gedaan door gebruik te maken van de response van de API om de `todos` -state bij te werken.

In een echte applicatie zou dit bijvoorbeeld automatisch kunnen gebeuren door de `GET` request opnieuw uit te voeren na een succesvolle `POST`, `PUT` of `DELETE` request, omdat de data op de server up-to-date is, zal de nieuwe response de nieuwe data bevatten en de `todos` -state correct bijwerken.

- Een `todos` -state die een lijst van taken (todo's) bijhoudt.
- Een `fetchTodos` -functie die de lijst van taken ophaalt via een `GET` request naar `https://jsonplaceholder.typicode.com/todos` en de `todos` -state bijwerkt.
- Een `addTodo` -functie die een nieuwe taak toevoegt via een `POST` request naar `https://jsonplaceholder.typicode.com/todos` en de `todos` -state bijwerkt.
- Een `updateTodo` -functie die een bestaande taak bijwerkt via een `PUT` request naar `https://jsonplaceholder.typicode.com/todos/{id}` en de `todos` -state bijwerkt.
- Een `deleteTodo` -functie die een taak verwijdert via een `DELETE` request naar `https://jsonplaceholder.typicode.com/todos/{id}` en de `todos` -state bijwerkt.
- Een `loading` -state die bijhoudt of er een API-call bezig is.
- Een `error` -state die eventuele fouten bijhoudt.

Component

Maak een Vue component `TodoList.vue` waarin een eenvoudige takenlijst (todo list) wordt geïmplementeerd met de volgende functionaliteiten:

- **Create:** Voeg een nieuwe taak toe door een `POST` request te sturen naar `https://jsonplaceholder.typicode.com/todos`.
- **Read:** Haal de lijst van taken op door een `GET` request te sturen naar `https://jsonplaceholder.typicode.com/todos`.
- **Update:** Werk een taak bij (bijvoorbeeld markeer als voltooid) door een `PUT` request te sturen naar `https://jsonplaceholder.typicode.com/todos/{id}`.
- **Delete:** Verwijder een taak door een `DELETE` request te sturen naar `https://jsonplaceholder.typicode.com/todos/{id}`.

`{id}` is de syntax om aan te geven dat het id van de taak moet ingevuld worden in plaats van `{id}`.

Om de nieuwe taak toe te voegen kan er een eenvoudig formulier gemaakt worden met een input en een knop. Bij het klikken op de knop wordt de `addTodo` -functie van de store aangeroepen met de ingevoerde taakgegevens.

Voor het bijwerken van een taak kan er bijvoorbeeld een checkbox gebruikt worden om aan te geven of de taak voltooid is. Bij het klikken op de checkbox wordt de `updateTodo` -functie van de store aangeroepen met de bijgewerkte taakgegevens.

Voor het verwijderen van een taak kan er een verwijderknop naast elke taak worden toegevoegd. Bij het klikken op de knop wordt de `deleteTodo` -functie van de store aangeroepen met het id van de taak die verwijderd moet worden.

De component moet ook de `loading` - en `error` -states van de store gebruiken om de gebruiker feedback te geven over de status van de API-calls.

View

Maak een Vue component `TodoView.vue` die de `TodoList` -component gebruikt en deze weergeeft op een specifieke route, bijvoorbeeld `/todos`.

Detail View

Voeg een functie `getTodoById(id)` toe aan de store om een specifieke taak op te halen via een `GET` request naar `https://jsonplaceholder.typicode.com/todos/{id}`, waarbij `{id}` het id van de taak is.

Maak een Vue component `TodoDetail.vue` die de details van een specifieke taak weergeeft. Deze component is enkel verantwoordelijk voor het tonen van de details van een taak, niet voor het ophalen van de data zelf. Er hoeft verder geen functionaliteit voor bijwerken of verwijderen toegevoegd te worden in deze component.

Maak een Vue component `TodoDetailView.vue` die de volgende functionaliteiten bevat:

- Haal de taakgegevens op basis van de `id` -parameter uit de route, hou deze bij in een `ref` -variable `todo`.
- Geef de `ref` door als prop aan de `TodoDetail` -component.
- Gebruik de `onMounted` lifecycle hook om de taakgegevens op te halen wanneer de component wordt gemount.

Props

Gebruik props om de taakgegevens van de `TodoList` -component door te geven aan de `TodoDetail` -component.

useRoute

Gebruik de `useRoute` -functie van `vue-router` om toegang te krijgen tot de routeparameters in de `TodoDetail.vue` -component.

```
import { onMounted } from "vue";
import { useRoute } from "vue-router";

const route = useRoute();

onMounted(() => {
  const todoId = route.params.id;
  // Geef de todoId door aan een store-functie om de
  // specifieke taak op te halen
});
```

Navigatie

Voeg navigatie toe tussen de `TodoListView` - en `TodoDetailView` - componenten. Vanuit de `TodoListView` kan er op een taak geklikt worden om naar de `TodoDetailView` te navigeren, waar de details van die specifieke taak worden weergegeven.

useRouter

Gebruik de `useRouter` -functie van `vue-router` om programmatisch te navigeren naar de `TodoDetailView` wanneer er op een taak in de lijst wordt geklikt.

```
import { useRouter } from "vue-router";

const router = useRouter();

function goToTodoDetail(id) {
  // Dit gaat er vanuit dat de "name" van de route voor
  // TodoDetailView "TodoDetail" is
  router.push({ name: "TodoDetail", params: { id } });
}
```

Bonus

Voeg een emit-event toe aan de `TodoDetail` -component die een signaal stuurt naar de `TodoDetailView` -component wanneer de gebruiker op een knop `Verwijderen` klikt in de `TodoDetail` -component. De `TodoDetailView` -component moet dan de `deleteTodo` -functie van de store aanroepen om de taak te verwijderen en vervolgens terug navigeren naar de `TodoListView` .

Pokédex Vue Edition

Over de verschillende hoofdstukken heen wordt een Pokédex-applicatie gemaakt. Elk hoofdstuk gebruikt de nieuwe kennis om de applicatie verder uit te breiden.

Accepteer de [GitHub Classroom Assignment](#) om te starten met de oefening.

De volledige opdracht is uitgewerkt in de README.md van de assignment.

Er wordt aangeraden om via `branches` te werken, elk onderdeel kan in een aparte branch gemaakt worden. De branch kan vervolgens via een Pull Request klaargezet worden voor feedback. Na feedback en goedkeuring kan de branch gemerged worden in `main` en kan de volgende oefening gestart worden in een nieuwe branch.

Verwerk voor dit hoofdstuk de onderdelen:

- API-integratie
 - SCSS
 - Extra functionaliteit Zoekfunctie
-

Previous page

Theorie

Next page

Theorie